

UASB03 – Projet pour le certificat de spécialisation Analyste de données massives

Projet: Prédiction de la consommation électrique française avec Spark et Cassandra.



Table des matières

Remerciements	2
Introduction :	3
Le contexte du projet.....	3
Les objectifs poursuivis	3
Démarche	4
Scalabilité	4
Analyse exploratoire sur un échantillon de données	4
Sélection et traitement des données climatiques en Grand Est :	5
Sélection et traitement des données de la consommation électrique en Grand Est	7
Analyse des données sélectionnées pour la région Grand Est	7
La prédiction future de la consommation avec AROME	9
Cassandra, schéma et stockage des données	10
Modèle logique	11
Modèle physique et implémentation du schéma.....	12
Spark et Cassandra pour un système distribué complet	14
L'association de Spark à Cassandra.....	15
La collecte et le stockage des données des capteurs climatiques	16
Sauvegarde des données dans Cassandra :	17
Création du modèle	17
Données AROME et application du modèle.....	21
Particularité de AROME et de son API	21
Traitement des cartes du modèle AROME	22
Application du modèle et comparaison avec les données réelles	23
Conclusion et pistes potentielles d'amélioration.....	25
Références.....	26

Remerciements

Je tiens à remercier tous les membres de l'équipe pédagogique qui ont élaboré le certificat de spécialisation '*Analyse de données massive*' du CNAM tant pour la qualité des enseignements que des contenus apportés, notamment Mme Ndèye Niang-Keita, M. Vincent Audigier, M. Philippe Rigaux et M. Michel Crucianu.

Un grand merci également aux équipes du CNAM Paris et du CNAM Grand Est pour les différents aspects logistiques.

Aussi, je souhaite également témoigner ma reconnaissance aux équipes de météo France, Open Data Réseaux Énergies et du Réseau de transport d'électricité, qui mettent et maintiennent à disposition des données publiques et qui ont répondu avec une très grande rapidité à certaines de mes demandes sur les API mises à disposition, ou tout bonnement sur l'intérêt de mon projet.

Pour finir, je souhaite remercier mes proches et collègues. Ils se reconnaîtront, car sans eux, ce projet n'aurait sans doute jamais vu le jour.



Métaphore de l'aspirateur par Marcel Lebrun et André Cerdu

Introduction :

Cette partie a pour but d'expliciter les motivations du projet, ces grandes lignes directrices ainsi que les deux objectifs poursuivis.

Le contexte du projet

La ressource énergétique électrique étant difficilement stockable, le gâchis électrique est difficilement envisageable et supportable financièrement.

La possibilité de prédire la consommation à l'échelle régionale, voire une échelle encore plus fine pourrait permettre d'anticiper au mieux et à l'avance la quantité d'énergie à produire et sa destination première.

Depuis janvier 2024, toutes les données publiques de Météo-France disposent d'une possibilité d'accès sans aucun frais et sont gratuitement réutilisables. Météo France met notamment à disposition gratuitement [via une API dédiée](#) les données climatologiques générées par des stations situées en métropole et outre-mer.

Ces centaines de stations génèrent différents types de données comme la température, la quantité et la durée des précipitations ou encore la force du vent à intervalle régulier (toutes les heures voire toutes les six minutes pour les données concernant les précipitations).

Ces données sont également historisées et il est possible de remonter plusieurs années auparavant afin d'obtenir des informations sur le climat dans différentes zones données.

Open Data Réseaux Énergies (ODRE) met pour sa part à disposition plusieurs jeux de données sur la consommation électrique en France. L'un des jeux de données présente les courbes de consommation régionale électrique (mis à jour toutes les 15 minutes en Mégawatt)

Ce jeu de données peut également être téléchargé [via une API](#) qui autorise la recherche selon différents paramètres. Les données sont également historisées et il est possible de récupérer la consommation électrique régionale depuis janvier 2023

Les objectifs poursuivis

Il existe déjà de nombreux travaux sur la prédiction de la consommation électrique, ce projet propose avec modestie une approche prenant appui sur les données mises à disposition du grand public.

Les principaux objectifs du projet sont donc les suivants :

- 1- Une étude de la corrélation entre les variables climatiques et la consommation électrique nationale et régionale française. Est-elle significative ? Est-ce que certaines variables climatiques ont un impact plus ou moins important sur la consommation électrique.

- 2- La création d'un modèle permettant de prédire à partir de différentes variables dont les variables climatiques générées par les stations la consommation électrique potentielle régionale française.

Démarche

Afin de répondre à ces différents objectifs, les étapes du projet ont été les suivantes :

1. Analyse d'un échantillon de données provenant des API de météo France et ODRE afin de confirmer qu'il existe un lien entre la consommation électrique et les variables des capteurs climatiques.
2. Identification des correspondances possibles entre les variables des capteurs climatiques et celles du modèle atmosphérique AROME.
3. Création de la base de données qui sera utilisée pour stocker les différentes variables, pour créer le modèle et l'appliquer aux données.
4. Création du modèle prédictif de la consommation et test des performances.
5. Récupération des données AROME et application du modèle prédictif.

Scalabilité

Au cours du projet, un simple pc a été utilisé, ce qui a entraîné certaines orientations pour le projet en lien avec les ressources machines disponibles :

- Une réduction de la focale sur une région (la région Grand Est) qui comporte plusieurs centaines de station climatique.
- Une certaine manière de traiter les cartes climatiques. En effet, les cartes climatiques récupérées depuis le site de météo France sont assez lourdes (plusieurs millions de lignes). Il était donc compliqué de les traiter et de répartir la charge de manière distribuée sur un seul nœud. Toutefois, certains commentaires accompagnent ce rapport sur la manière de mieux distribuer certaines opérations dans un cluster afin de tirer parti de la distribution des données et des calculs dans Spark, et surtout, éviter des goulots d'étranglement.
- Un modèle avec des résultats plus que louables mais dont les performances pourraient être augmentés dans un cluster distribué. Nous y reviendrons dans la conclusion de ce rapport.

Analyse exploratoire sur un échantillon de données

Le premier objectif du projet était de confirmer qu'il y avait un lien entre la consommation électrique et certaines variables climatiques, dans cette optique, des données journalières ont été récoltées dans un premier temps sur une année pour la région Grand Est.

Cette partie a pour but de répondre à cette question.

Sélection et traitement des données climatiques en Grand Est :

Les données climatiques ont été collectées au niveau d'une centaine de station météo réparties dans toute la région Grand Est via [l'API données climatologiques de météo France](#).

A savoir que 5 types de station récoltent des données en France, leurs caractéristiques sont les suivantes :

Type de station		Disponibilité des données
0	Station synoptique, acquisition temps réel, expertise à J+1	Horaire
1	Station automatique Radome-Resome, acquisition temp réel, expertise à J+1	
2	Station automatique NON Radome-Resome, acquisition temp réel, expertise à J+1	
3	Station automatique, acquisition temps réel, expertise temps différé (à M+21 jours max)	Vers le 20 du mois pour les données du mois précédent
4	Poste climatologique manuel ou station automatique, acquisition temps différé, expertise temps différé (à M+21 jours max)	
5	Station avec acquisition temps réel ou différé, non expertisée ou expertise des données non garantie	Occasionnelle

Dans le cadre du projet, ce sont les stations de type 2 qui ont été choisies car elles :

- Sont plus nombreuses et bien réparties sur le territoire français.
- Enregistrent des données à l'échelle journalière et horaire.

Les variables récupérées sur ces stations sont les suivantes :

- RR : Quantité de précipitation tombée en 24 heures.
- TN : Température minimale sous abri.
- HTN : Heure de TN (hhmm).
- TX : Température maximale sous abri.
- HTX : Heure de TX (hhmm).
- TM : Moyenne quotidienne des températures horaires sous abri.
- TMNX : Moyenne quotidienne (TN+TX)/2.
- DG : Durée de gel sous abri.
- TAMPLI : Amplitude thermique quotidienne (TX-TN).
- TNTXM : Moyenne quotidienne (TN+TX)/2 arrondie supérieur.
- ETPGRILLE : Evapotranspiration calculée au point de grille le plus proche (en millimètre)
- NEIG : Occurrence de neige quotidienne : (vaut 0 si pas de neige et 1 sinon)

```

for station in stations:
    id_station = station.get("id")
    #On vérifie préalablement que le fichier n'existe pas déjà avec le if, si ce n'est pas le cas, on lance la commande
    if id_station not in liste_mesure_stations :
        try :
            response = self.request('GET', 'https://public-api.meteofrance.fr/public/DPCLim/v1/commande-station/quotidienne?id-station='+id_station+'&date-deb-periode='
            #print(response.status_code)
            response_json = json.loads(response.content)
            # On récupère l'id de la commande (c'est une API asynchrone)
            id_commande = response_json.get('elaboreProduitAvecDemandeResponse').get('return')
            time.sleep(3)
            #On récupère les données avec la commande de la station cible:
            fichier_recup = self.request('GET', 'https://public-api.meteofrance.fr/public/DPCLim/v1/commande/fichier?id-cmde='+id_commande,verify=False)
            #print(fichier_recup.status_code)
            filename='Stations/GE/Mesures/'+id_station+'.csv'
            with open(filename, 'wb') as file:
                file.write(fichier_recup.content)
                print("Fichier pour station "+id_station+" créé")
        except :
            # Capturer des erreurs spécifiques comme ZeroDivisionError ou TypeError
            print("Une erreur est apparue avec la station : "+id_station)

```

Extrait du code '0- Exploration d'un échantillon de données.ipynb ' qui permet de récupérer les données des stations météo à l'aide de l'API asynchrone de météo France

Les données récoltées avaient une certaine quantité de valeurs manquantes, plusieurs traitements ont donc été effectués :

- 1- Les stations dont toutes les variables avaient beaucoup de valeurs manquantes ont été écartées de l'analyse.
- 2- Les variables avec beaucoup de données manquantes et qui n'étaient pas communes à la majorité des stations ont été écartées de l'analyse.
- 3- Les autres variables des stations avec peu de données manquantes ont été imputées à l'aide de la méthode des k plus proches voisins (A noter que cette approche d'imputation ne sera pas utilisée par la suite, en effet, l'imputation par la médiane à l'aide de l'estimateur d'imputation de Spark sera privilégié pour mieux tirer parti de la distribution des opérations.

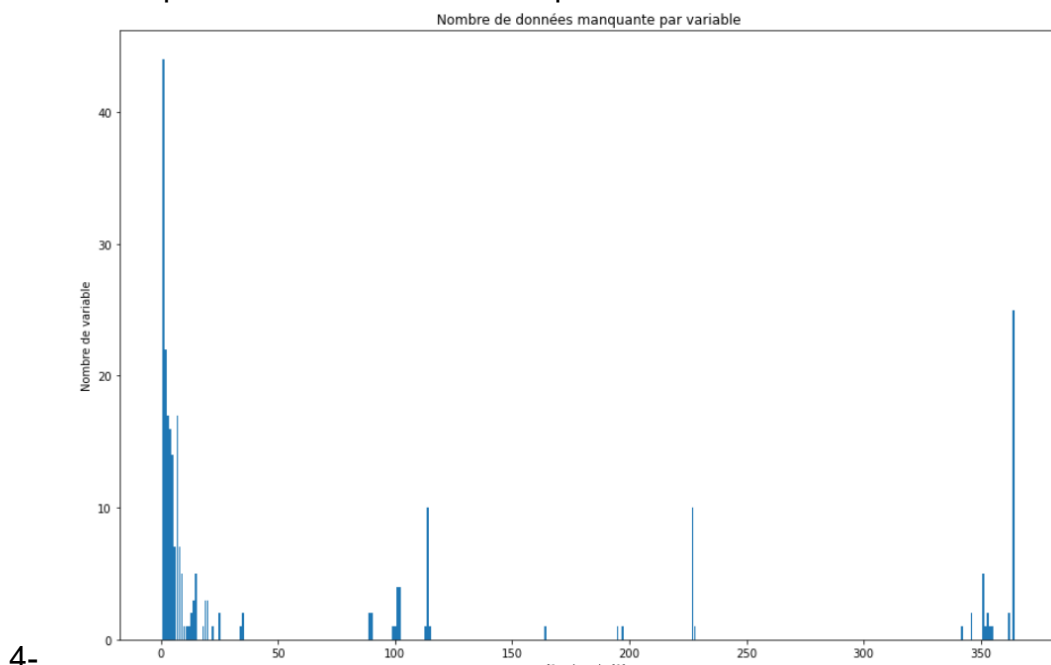


Diagramme à barres des variables avec le nombre de NA sur 365 jours avant imputation, ici, plusieurs dizaines de variable ont plus de 350 valeurs manquantes sur 365.

Sélection et traitement des données de la consommation électrique en Grand Est

Les données sur la consommation électrique ont été récupérées [via l'API sur la consommation électrique régionale.](#)

Dans le cadre du projet et des objectifs fixés, ce sont les 3 variables suivantes qui ont été choisies :

Nom de la variable	Signification
Région	Nom de la région
Date - Heure	Date et heure au format : 2023-08-28T09:45
Consommation (MW)	Consommation de la région sur les 15 dernières minutes en MegaWatt.

Ces variables ont été regroupées afin d'obtenir des mesures à l'échelle journalière dans le but de les joindre ensuite aux données climatiques.

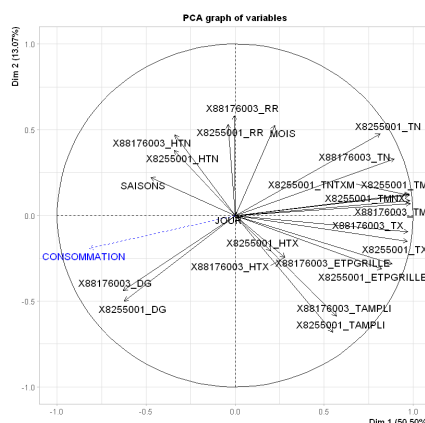
Analyse des données sélectionnées pour la région Grand Est

Les données des stations climatiques et de la consommation électrique ont ensuite été jointes dans un Dataframe de dimension (365, 1248).

Ce Dataframe comprend donc 365 observations journalières de toutes les variables mesurées par les stations météo de type 2 en région Grand Est (1248).

L'analyse de la corrélation a été réalisée en deux temps :

- 1- Une matrice de corrélation de Pearson a été créée afin d'identifier les variables corrélées.
- 2- Une analyse en composante principale a été réalisée avec FactorMineR, dans un premier temps sur le dataframe de dimension (365,1248), puis afin d'obtenir plus de visibilité sur des dataframes plus petits dans le but de mieux interpréter le cercle des corrélations

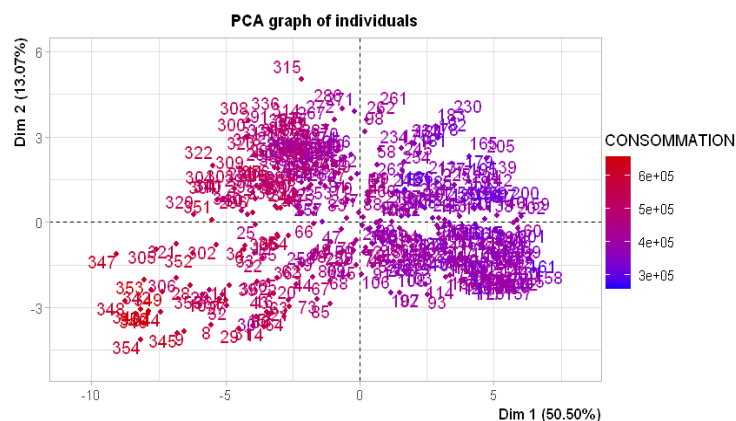


*Un cercle des corrélations obtenu avec FactorMineR
sur les variables de plusieurs stations climatiques*

Ci-dessus, le cercle de corrélation sur plusieurs stations permet de visualiser sur la première dimension que la consommation électrique est corrélée négativement aux variables températures (TN, TM, TX, ...), cette même variable étant en revanche assez corrélée à la variable gel (DG). On peut donc en déduire que la dimension1 concerne les changements de température, plus la température augmente et plus la consommation diminue, tandis que plus la température diminue et plus la consommation électrique augmente.

La dimension 2 met en opposition la quantité de précipitation (RR) et l'amplitude thermique (TAMPLI).

Cette confirmation de l'opposition entre température et consommation a été réalisée en regardant le graphe des individus ainsi que la matrice corrélation de Pearson.



Le graphe des individus avec la variable consommation apparaît en couleur

Sur le graphe ci-dessus, les individus avec une consommation forte sont à gauche, ils ont généralement des températures faibles tandis qu'à droite les températures sont de plus en plus élevées alors que la consommation diminue.

La matrice de corrélation réalisée plus tôt confirme cette analyse, les variables de température des stations ont une assez forte corrélation négative (de -0.79 à -0.82) à la variable consommation électrique. La variable gel a une corrélation positive (entre 0.6 et 0.7) à cette même variable.

En somme, la variable température joue un rôle assez significatif sur la consommation électrique, même si celle-ci n'est pas totalement linéaire.

Cette première approche sur un échantillon était nécessaire. Elle m'a permis de :

- 1- Observer qu'une corrélation négative existe avec les températures mais aussi que d'autres variables sont corrélées positivement à la consommation comme le gel ou encore la saison.
- 2- Orienter la manière de stocker les données grâce à une meilleure visibilité des données mesurées, des formats de fichiers envoyés par les API et des différentes opérations qui devront être exécutées.
- 3- D'autre part, elle m'a également permis de découvrir le fonctionnement des API de météo France et des limitations techniques sur le nombre de requête envoyé par minute.

La prédiction future de la consommation avec AROME

La liste des variables récupérée dans la partie précédente permet de bien mesurer le lien entre variables climatiques et consommation électrique.

On peut retenir que les variables des températures ont un impact assez significatif alors que d'autres variables ont un impact moindre.

S'il est possible de créer un modèle prédictif sur la base de certaines variables des stations climatiques, celui-ci sera en mesure de prédire la consommation à l'instant T, en revanche, la prédiction de la consommation à T+1h sera impossible car les variables des capteurs climatiques seront tout bonnement indisponibles.

Une alternative, AROME, système régional de prévision numérique développé par le CNRS couvre la France métropolitaine et permet de récupérer des prévisions climatiques avec une maille fine d'environ 1,25km. Sa conception intègre de nombreuses données telles que celles des stations automatiques au sol, radiosondages, stations GTPS, satellites, ...).

Si les variables des capteurs climatiques sont indisponibles sur les 24 prochaines heures, il est en revanche possible de récupérer les prédictions climatiques du modèle AROME et de les associer aux stations climatiques en fonction des coordonnées GPS. Sur la base de ce constat, un croisement entre les variables des capteurs climatiques et ceux du modèle AROME a été effectué afin de visualiser les correspondances possibles et les variables sur lesquelles se focaliser :

Capteurs climatiques	AROME 0.01	AROME 0.025
T : température sous abri instantanée (en °C et 1/10)"	T (2m) : correspond à la température à 2 m de hauteur au-dessus du relief. (Peut être comparé aux mesures faites in situ par les stations d'observations humaine ou automatique) - En Kelvin	T (2m) : Correspond à la température à 2 m de hauteur au-dessus du relief. (Peut être comparé aux mesures faites in situ par les stations d'observations humaine ou automatique) - En Kelvin
TX : température maximale sous abri dans l'heure (en °C et 1/10)	TMAX (2m) : Maximum des températures diagnostiquée à 2m. (Peut être comparé aux mesures faites in situ par les stations d'observations humaine ou automatique) - En Kelvin	Pas d'équivalent
HTX : heure de TX (hhmm)	Pas d'équivalent	Pas d'équivalent
TN : température minimale sous abri	TMIN (2m) : Minimum des températures	Pas d'équivalent

dans l'heure (en °C et 1/10)	diagnostiquée à 2m (Peut être comparé aux mesures faites in situ par les stations d'observations humaine ou automatique) - En Kelvin	
HTN : heure de TN (hhmm)	Pas d'équivalent	Pas d'équivalent
RR1 : quantité de précipitation tombée en 1 heure (en mm et 1/10 mm)	EAU (Difficile à traiter et sera donc écartée) Car c'est le cumul (depuis le début de la simulation) de la somme des flux au sol des précipitations liquides issues des processus stratiforme.	Pas d'équivalent
DG : durée de gel sous abri	Pas d'équivalent	Pas d'équivalent

Pour la suite du projet, le choix de se tourner vers le modèle AROME 0.025 et de se focaliser principalement sur la température sous abri instantané a été fait.

Il aurait été possible de s'orienter vers l'obtention de plus de données avec les 3 variables températures des stations (T, TX et TN) qui ont un équivalent assez clair dans AROME (T, TMAX et TMIN) avec le modèle 0.01, toutefois, le temps imparti m'a fait prendre certaines orientations aux projets. Nous y reviendrons dans la conclusion avec certaines pistes d'amélioration possible à ce projet.

Cassandra, schéma et stockage des données

Cassandra est une base de données distribuée de type NOSQL. Elle a été choisie et est utilisée dans le cadre de ce projet pour les raisons suivantes :

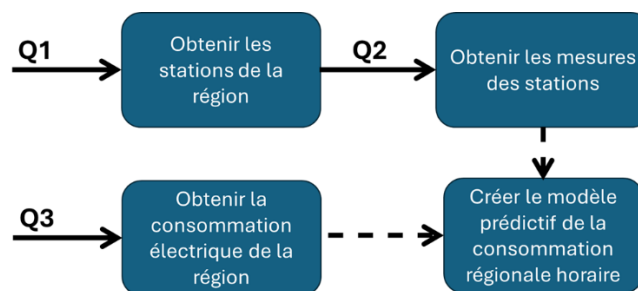
1. La structure des données à enregistrer vise à peu évoluer (données provenant des capteurs climatiques, consommation électrique, données des modèles atmosphériques, ...), et donc, cette structure correspond bien à la philosophie de Cassandra qui est basé sur la création de schéma.
2. Cassandra permet de passer à l'échelle en distribuant les données sur plusieurs nœuds basée sur le consistant hashing pour le partitionnement et la réplication. Le simple historique des mesures des stations climatiques à l'échelle horaire ou des prévisions climatiques AROME peut rapidement devenir très important.
3. Sur des aspects plus pragmatiques, il est rapidement possible d'interagir avec les données de la base avec le langage CQL et l'outil DBVisualizer.

Modèle logique

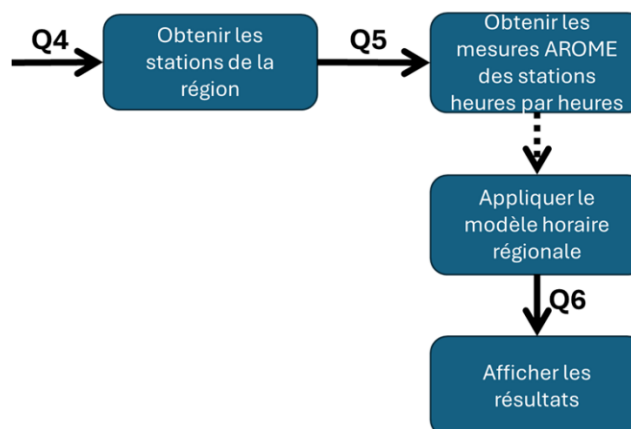
Pour créer la base de données, il faut en déterminer le schéma. Afin de réaliser un schéma Cassandra, il est nécessaire de lister l'ensemble des requêtes qui seront utilisées dans le but de déterminer ensuite les tables à créer. On doit donc identifier la séquence des requêtes transmises. Les tables doivent être organisées pour que chaque requête s'exécute localement avec un critère d'accès portant sur la clef.

Le besoin, dans le cadre de la prédiction de la consommation consiste dans un premier temps à :

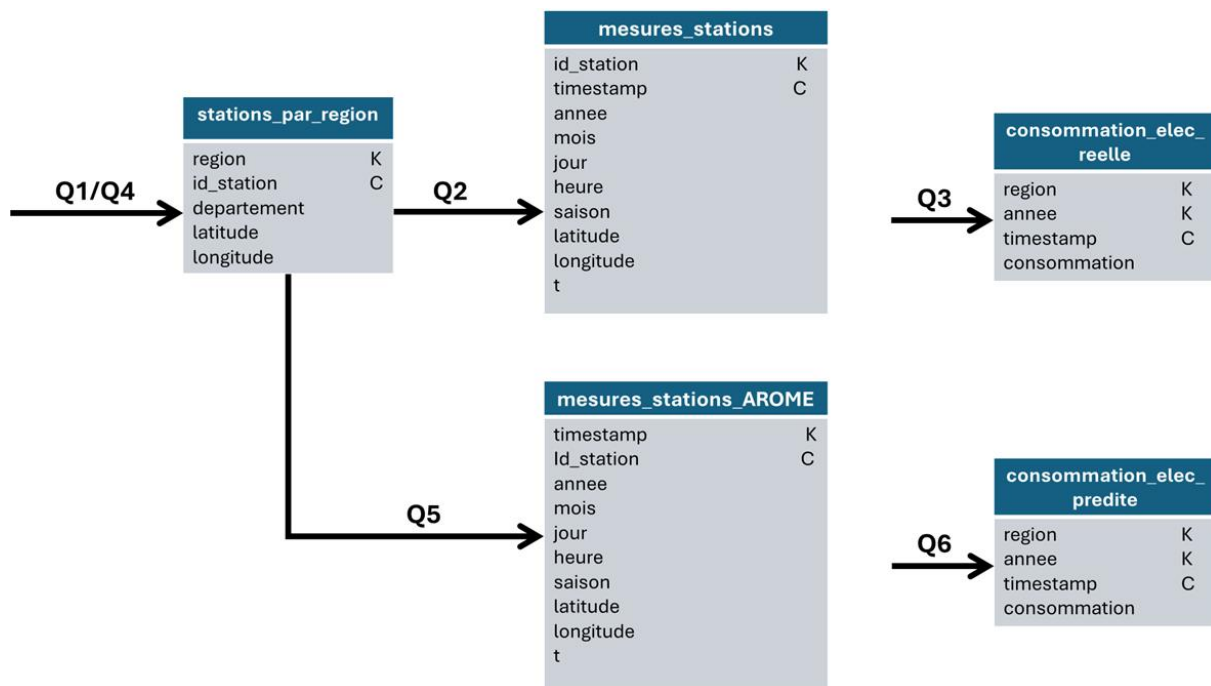
1. Créer le modèle de prédiction à partir des données des capteurs de la région concernée. Les requêtes nécessaires sont les suivantes :
 - Q1 : Récupérer les numéros des capteurs climatiques de la région
 - Q2 : Récupérer les mesures des variables climatiques réelles des capteurs de la région
 - Q3 : Récupérer la consommation électrique de la région



2. Prédire la consommation de la région à partir du modèle créé. Les requêtes sont les suivantes :
 - Q1/Q4 : Récupérer les numéros des capteurs climatiques de la région
 - Q5 : Récupérer les variables climatiques prédites des capteurs sur une heure à l'aide du modèle AROME
 - Q6 : Récupérer la consommation électrique prédite de la région



Sur la base de ces requêtes, nous pouvons définir le modèle suivant :



Les attributs de notre schéma ci-dessous qui comportent un K et un C, sont des attributs de la clé :

1. K correspond aux attributs de partitionnement et déterminent les fragments à créer et le placement des lignes sur le serveur. A savoir qu'une requête incluant comme critère une clé de partitionnement ne concerne qu'un seul serveur.
- C correspond aux attributs de regroupement qui permettent de trier les lignes sur le serveur.

Les requête Q1/Q4 et Q2 et Q5 sont des requêtes qui sont séquencées. La finalité étant de rechercher les mesures des stations sur la base de l'identifiant des capteurs de la région concernée.

Pour la requête Q3, celle-ci sert à récupérer les données provenant de l'API mis à disposition par le gestionnaire du réseau de transport d'électricité.

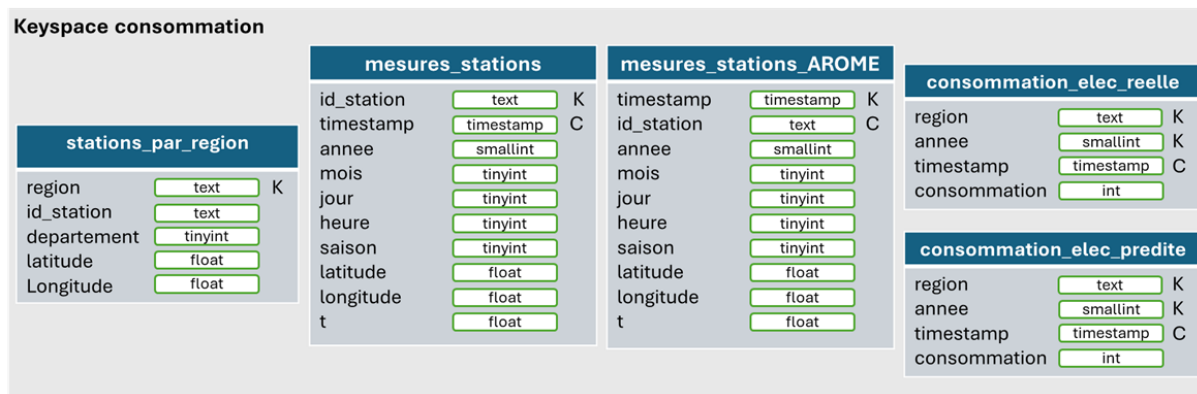
Pour rappel, notre programme doit joindre ces données aux mesures climatiques afin de créer le modèle prédictif de la consommation horaire.

La requête Q6, quant à elle, est destinée à récupérer les prédictions réalisées par notre modèle afin de pouvoir les afficher dans un graphique linéaire.

Modèle physique et implémentation du schéma

Sur la base de notre modèle logique, nous pouvons déterminer le modèle physique puis les requêtes CQL qui permettront d'implémenter le schéma.

Le modèle étant relativement simple, un simple Keyspace (consommation) sera créé et contiendra l'ensemble de nos tables :



La liste des commandes CQL suivantes permettent d'implémenter notre schéma dans Cassandra :

```
CREATE KEYSPACE consommation WITH replication =
{'class': 'SimpleStrategy', 'replication_factor': 3};
```

La commande ci-dessus permet de créer le keyspace avec la stratégie de réplication associée. Une réplication de 3 a été choisie. La réplication permet de copier les données dans le but de palier les pannes potentiels des nœuds du cluster et, ainsi, d'assurer au maximum d'avoir les données toujours à disposition. Cette réplication est possible via le paramètre '*Replication_factor*' qui détermine le nombre de copie à créer. Il est considéré que 2 à 3 copies constituent un bon niveau de sécurité, évidemment, cette option nécessite d'autres nœuds dans le cluster afin de pouvoir déplacer les copies dans ceux-ci.

Nous pouvons ensuite créer nos différentes tables avec Cassandra Query Language :

```
CREATE TABLE stations_par_region (
  region text,
  id_station text,
  departement tinyint,
  latitude double,
  longitude double,
  PRIMARY KEY (region, id_station));
```

```
CREATE TABLE mesures_stations (
  Id_station text,
  timestamp timestamp,
  annee smallint,
  mois tinyint,
  jour tinyint,
  heure tinyint,
  saison tinyint,
  t float,
  PRIMARY KEY (id_station, timestamp)
  WITH CLUSTERING ORDER BY (timestamp ASC);
```

```
CREATE TABLE mesures_stations_AROME (  
  timestamp timestamp,  
  id_station text,  
  annee smallint,  
  mois tinyint,  
  jour tinyint,  
  heure tinyint,  
  saison tinyint,  
  t float,  
  PRIMARY KEY (timestamp, id_station))
```

```
CREATE TABLE consommation_elec_reelle (  
  region text,  
  annee smallint,  
  timestamp timestamp,  
  consommation int,  
  PRIMARY KEY ((region, annee), timestamp))  
WITH CLUSTERING ORDER BY (timestamp ASC) ;
```

```
CREATE TABLE consommation_elec_predite (  
  region text,  
  annee smallint,  
  timestamp timestamp,  
  consommation int,  
  PRIMARY KEY ((region, annee), timestamp))  
WITH CLUSTERING ORDER BY (timestamp ASC) ;
```

Spark et Cassandra pour un système distribué complet

Spark est un Framework de développement de calculs distribués qui permet de traiter et d'analyser d'énormes quantités de données, qu'elles soient non-structurées, semi-structurées et structurées au sein d'un cluster de machines.

Si Cassandra est dédiée au stockage et au partitionnement des données, Spark, quant à lui, servira à les manipuler et à effectuer toutes les opérations nécessaires de manière distribuée.

Spark possède différentes API et bibliothèques, dans notre cas, étant sur des données principalement structurées, les APIs de haut niveau dédiées à manipuler les données structurées avec un schéma ou encore certaines bibliothèques comme celles sur l'apprentissage automatique seront utilisées.

Il existe plusieurs concepts fondamentaux dans Spark, notamment :

Le RDD (Résilient Distributed Dataset) :

- Résilient : Si une panne intervient, il est possible de reconstituer le RDD

- Distributed : Les données sont distribuées dans le cluster afin de partager les différentes opérations dans ses nœuds.
- Datasets : Ensemble de données de types potentiellement distinct.

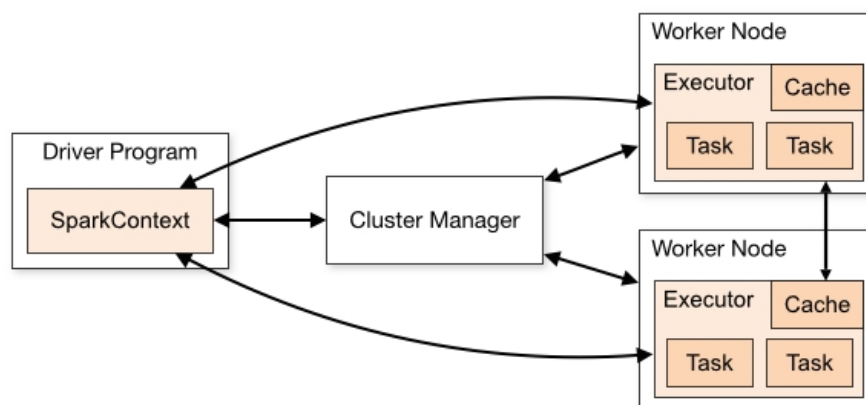
Le DAG (Directed Acyclic Graph) :

Ce sont l'ensemble des opérations que l'on va demander à Spark de réaliser sur nos RDD. Elles sont de deux types :

1. Les transformations qui permettent de manipuler les données mais sans les exécuter. Cela rejoint le principe de fonctionnement lazy/paresseux de Spark.
2. Les actions qui permettent d'exécuter les transformations et donc de lancer les calculs.

Lors du lancement d'une application dans Spark par un client :

1. Un driver va être lancé pour cette application dont la tâche est de demander les ressources nécessaires au cluster manager afin de distribuer les différentes opérations du DAG à des nœuds (les workers) mais aussi de créer les RDD.
2. Les workers vont créer des executors qui vont effectuer les opérations prévues du DAG selon les directives du driver et cela de manière distribuée.
3. Une fois les opérations terminées, les résultats sont ensuite retournés.



Source : <https://spark.apache.org/docs/latest/cluster-overview.html>

L'association de Spark à Cassandra

L'association de Cassandra avec Spark est réalisé avec le connecteur dédié '*spark-cassandra-connector*' de l'entreprise Datastax.

Afin de configurer au mieux cette association, il faut bien prendre comprendre qu'une partition Cassandra est différente d'une partition Spark :

1. Une partition Cassandra est une collection (une table) qui est divisée en fragments. Cette division est déterminée à l'aide de la clef de partitionnement choisie lors de la création des tables de la base de données.

2. Une partition Spark est un RDD qui est divisé en fragments. Ces fragments sont distribués dans les workers du cluster pour effectuer l'ensemble des opérations du DAG de manière distribuée.

Par défaut, Spark Cassandra Connector crée des fragments sparks qui sont alignés aux clefs de partition de Cassandra, toutefois, si l'on veut une bonne correspondance entre ces fragments et leur taille ainsi qu'un bon parallélisme, il est possible de paramétrer plusieurs choses :

1. Spark et Cassandra peuvent cohabiter dans un même cluster ou être dans des clusters différents. Dans le premier cas, la cohabitation dans le même environnement permet de meilleures performances en tirant partie de la localité des données et de la faible latence. Ce fonctionnement est plus approprié dans le cas où des lectures et écritures sont courantes. De plus la localité des données permette de diminuer les shuffles pouvant être produits par certaines actions.
2. Les workers et leurs executors doivent être en quantité suffisante pour gérer les fragments. Si les calculs à réaliser en parallèle concernent des tables partitionnées en centaines de fragments, les executors doivent avoir le nombre adéquat. Une trop faible quantité ne permettrait pas une distribution des opérations de manière optimale.

La connexion entre les deux systèmes s'effectue avec le code suivant :

```
session = SparkSession.builder \  
    .appName("consommation_electrique") \  
    .config("spark.cassandra.connection.host", "localhost") \  
    .config("spark.cassandra.connection.port", "3000") \  
    .config("spark.sql.extensions", "com.datastax.spark.connector.CassandraSparkExtensions") \  
    .getOrCreate()
```

Et le chargement d'une table peut tout simplement s'effectuer avec l'objet session :

```
stations_par_region = session.read.format("org.apache.spark.sql.cassandra") \  
    .options(table="stations_par_region", keyspace="consommation") \  
    .load()  
stations_par_region()
```

La collecte et le stockage des données des capteurs climatiques

Comme le projet a pour but de prédire la consommation électrique à l'échelle horaire, celles-ci ont été récupérées d'une manière assez similaire que les données journalières, globalement :

- Les données horaires ont été collectées via l'API asynchrone de météo France, seul le critère de sélection de l'échelle de temps change.
- Les données sur la consommation électrique ont été collectées à l'échelle 15 minutes puis regroupées à une échelle horaire dans la table dédiée
- Une réduction des données en colonne a été réalisée de la même façon que dans la partie précédente, c'est-à-dire que les stations avec une grande quantité de données manquantes ainsi que les variables non homogènes entre les stations ont été écartées
- Une imputation sur les stations dont les données manquantes sont faibles a été réalisée, en revanche, les stations comportant de nombreux jours manquants ont été écartées. Ce choix a été réalisé car l'imputation par la médiane de spark qui a été utilisée est perfectible et peut introduire des erreurs.
- Certaines variables ont été ajoutées comme la variable heure qui permet d'indiquer l'heure, la variable mois, le jour ou encore la saison.

Sauvegarde des données dans Cassandra :

L'historique des enregistrements des stations climatiques et de la consommation électrique a été sauvegardée dans la base NoSQL Cassandra à l'aide de Spark et du pilote DataStax Connector.

```
#On effectue l'ensemble des opérations avant écriture dans la base de données :
dataf = dataf.withColumn('saison', UDFreturn_saison(month(dataf["DATE"]))).withColumn('mois', month(dataf["DATE"])).withColumn('jour', dayofweek(dataf["DATE"])).withColumn('HEURE', hour(dataf["DATE"])).withColumnRenamed('POSTE', 'idstation').withColumn('region', lit(region))
dataf = dataf.withColumnRenamed('DATE', 'date_mesure')
dataf = dataf.select('idstation', 'region', 'date_mesure', 'saison', 'mois', 'jour', 'heure', 'rr1', 't', 'tn', 'htn', 'tx', 'htx', 'dg')
print(dataf.schema)
#On écrit dans la base :
try :
    dataf.write.format("org.apache.spark.sql.cassandra").options(table="mesures_des_stations", keyspace="consommation_elec_0") \
        .mode("append") \
        .save()
```

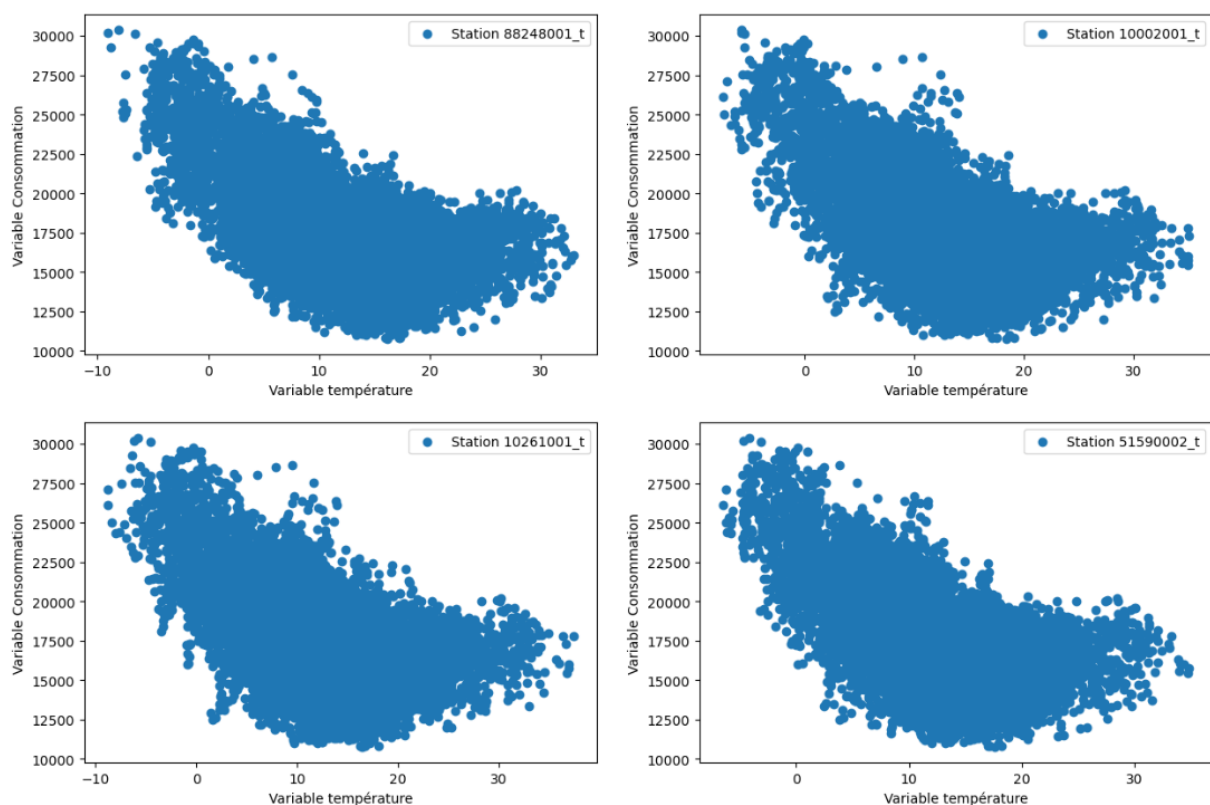
Extrait du code '1- Sélection des données, pré-traitement et stockage dans Cassandra.ipynb' qui permet de stocker les mesures des stations dans une table Cassandra

Création du modèle

Les données ont été regroupées dans un Dataframe de dimension (8736, 797) qui correspond à 8736 observations horaires sur l'année 2024 concernant les variables des stations de la région Grand Est (797). D'autres analyses ont été effectuées comme des analyses de corrélation entre les variables horaires et la consommation ou encore des diagrammes de dispersion.

0.89	0.94	0.9	0.89	0.88	1	0.88	0.93	0.95	0.95	0.9	0.91	0.9	0.94	0.9	0.9	0.91	0.91	0.92	0.89	0.89	0.61
0.97	0.95	0.98	0.99	0.96	0.88	1	0.96	0.95	0.95	0.97	0.97	0.97	0.95	0.97	0.97	0.97	0.97	0.96	0.96	0.96	0.61
0.98	0.97	0.97	0.96	0.97	0.93	0.96	1	0.97	0.97	0.96	0.97	0.97	0.97	0.96	0.96	0.97	0.97	0.97	0.96	0.95	0.61
0.95	0.98	0.96	0.96	0.93	0.95	0.95	0.97	1	0.95	0.95	0.96	0.95	0.96	0.95	0.95	0.95	0.96	0.93	0.93	0.65	
0.96	0.96	0.96	0.94	0.96	0.95	0.95	0.97	0.95	1	0.96	0.96	0.96	0.99	0.96	0.96	0.96	0.97	0.97	0.97	0.95	0.58
0.96	0.96	0.97	0.97	0.96	0.9	0.97	0.96	0.95	0.96	1	0.99	0.99	0.97	0.99	0.99	0.99	0.98	0.98	0.97	0.99	0.6
0.96	0.96	0.97	0.97	0.96	0.91	0.97	0.97	0.96	0.96	0.99	1	0.99	0.97	0.99	0.99	0.99	0.98	0.99	0.97	0.98	0.61
0.96	0.96	0.97	0.97	0.96	0.9	0.97	0.97	0.95	0.96	0.99	0.99	1	0.97	0.99	0.99	0.99	0.99	0.97	0.98	0.6	
0.96	0.97	0.97	0.95	0.97	0.94	0.95	0.97	0.96	0.99	0.97	0.97	0.97	1	0.97	0.97	0.98	0.98	0.98	0.98	0.58	
0.97	0.95	0.97	0.96	0.97	0.9	0.97	0.96	0.95	0.96	0.99	0.99	0.99	0.97	1	0.99	0.99	0.99	0.98	0.98	0.59	
0.97	0.96	0.97	0.97	0.96	0.9	0.97	0.96	0.95	0.96	0.99	0.99	0.99	0.97	0.99	1	0.98	0.98	0.97	0.98	0.62	
0.97	0.96	0.97	0.97	0.97	0.91	0.97	0.97	0.95	0.96	0.99	0.99	0.99	0.98	0.99	0.98	1	0.99	0.99	0.98	0.59	
0.97	0.96	0.97	0.97	0.97	0.91	0.97	0.97	0.95	0.97	0.98	0.98	0.99	0.98	0.99	0.98	0.99	0.99	1	0.98	0.59	
0.96	0.96	0.97	0.96	0.97	0.92	0.96	0.97	0.96	0.97	0.98	0.99	0.99	0.98	0.99	0.98	0.99	0.99	0.99	1	0.98	0.59
0.96	0.95	0.97	0.95	0.98	0.89	0.96	0.96	0.93	0.97	0.97	0.97	0.97	0.98	0.98	0.97	0.98	0.99	0.98	1	0.97	0.56
0.95	0.94	0.96	0.95	0.96	0.89	0.96	0.95	0.93	0.95	0.99	0.98	0.98	0.96	0.98	0.98	0.98	0.97	0.98	0.97	1	0.57
-0.6	-0.61	0.59	0.63	0.54	0.61	0.61	0.61	0.65	0.58	-0.6	-0.61	-0.6	-0.58	0.59	0.62	0.59	0.58	0.59	0.56	0.57	1
68188002	68190001	68226005	68231001	68247001	68249002	68287001	68307001	68338001	88075005	88095003	88147001	88176003	88196009	88201003	88248001	88340001	88383001	88399001	88408002	88430001	consommation

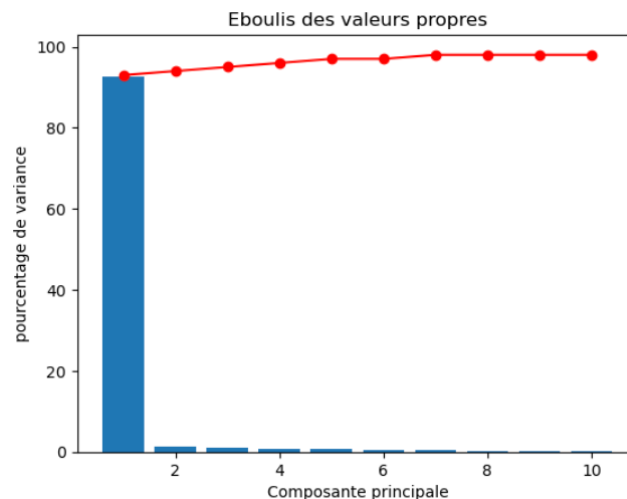
Matrice de corrélation entre les stations au pas de temps d'une heure et la consommation
Nous pouvons voir que les variables sont assez anti-corrélées (entre -0.55 et -0.65)



Les diagrammes sur quelques stations ci-dessus nous montre assez clairement que le lien entre la consommation et la température n'est pas totalement linéaire.

Une normalisation des données puis une réduction des dimensions a été réalisée via une analyse en composante principale.

Sur le graphique ci-dessous on peut voir que L'ACP capture 90% de la variance sur les 2 premières composantes.



Le jeu de données a été découpé en deux échantillons (un échantillon d'entraînement et un de test) afin d'entraîner différents modèles avec le jeu d'entraînement et tester les performances de notre modèle sur le jeu de test.

```
#1- Vecteur assembleur :
assembleur = VectorAssembler().setInputCols(listecolonne).setOutputCol("features")
# Construction du Dataframe mesures en appliquant le VectorAssembler
mesures = assembleur.transform(tableau_final_2).select('timestamp', 'features', 'label').cache()
# Partitionnement des données en un jeu d'apprentissage (80%) et un jeu de test (20%)
partitions = mesures.randomSplit([0.8, 0.2], seed=10)
apprentissage = partitions[0] # conservé en mémoire
test = partitions[1]
```

Création du jeu d'entraînement et du jeu de test

La régression linéaire et les forêts aléatoires ont été utilisés pour créer le prédicteur et ses performances ont été mesurées avec l'écart quadratique moyen (RMSE) et l'erreur moyenne absolue en pourcentage (MAPE).

```
[14]: paramGridR = ParamGridBuilder() \
        .addGrid(pca.k, [5]) \
        .addGrid(rf.numTrees, [120]) \
        .addGrid(rf.maxDepth, [12]) \
        .build()

pipelineR = Pipeline().setStages([scaler, pca, rf])

cvR = CrossValidator().setEstimator(pipelineR) \
        .setEstimatorParamMaps(paramGridR) \
        .setNumFolds(5) \
        .setEvaluator(RegressionEvaluator(metricName="rmse"))

[15]: rfmodelA = cvR.fit(apprentissage)
        rfmodelA.avgMetrics

[15]: [1266.232298403687]

[16]: # Calcul des résultats de test
        resultatsR = rfmodelA.transform(test)

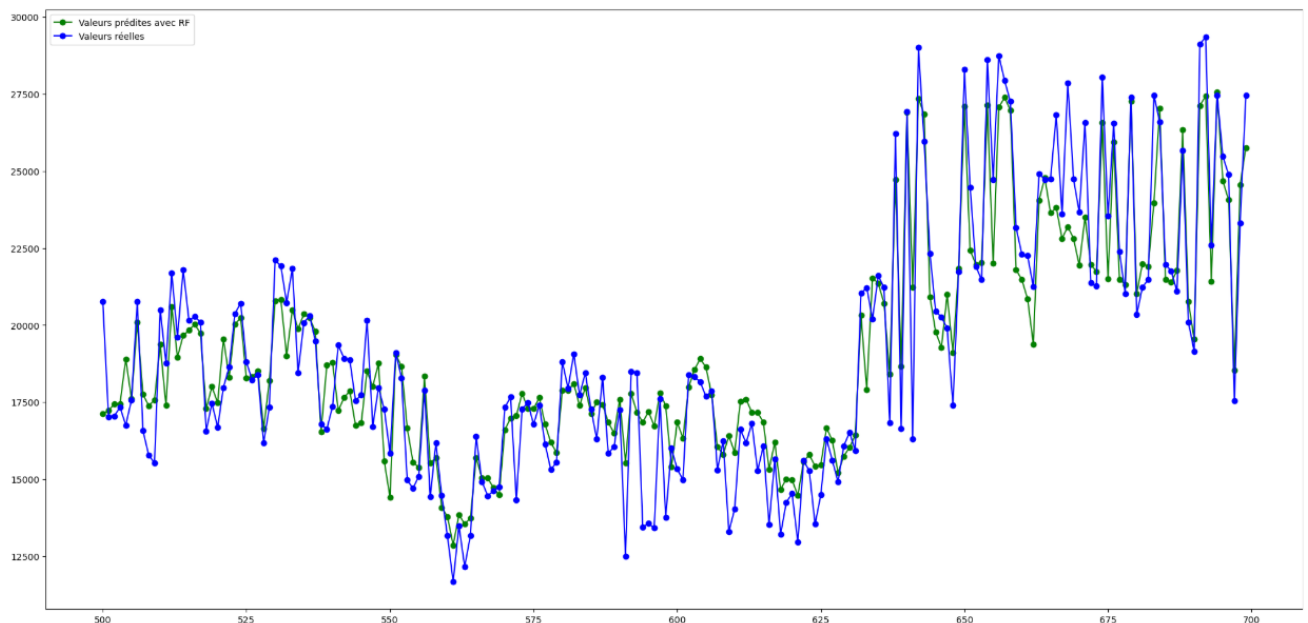
        # Évaluation des métriques de test
        rfmodelA.getEvaluator().evaluate(resultatsR)

[16]: 1240.3737422144593
```

Pipeline pour créer le prédicteur avec une évaluation par RMSE

Les forêts aléatoires ont obtenu rapidement de meilleures performances que la régression linéaire, ce qui n'est pas si étonnant car nous avons déjà identifié que le lien entre les variables climatiques et la consommation électrique n'est pas complètement linéaire.

Modèle	Paramètres	RMSE (sur le jeu de test)	MAPE (sur le jeu de test)	Remarques
Régression linéaire	Composantes : 5 RegParam : 0.3 ElasticNetParam : 0.6 maxIter : 200	~2200	~10,2%	Faute de ressource machine, certains paramètres comme le nombre de composantes n'ont pas pu être augmentés. Le paramétrage actuel peut donc éliminer de l'information utile et réduire les performances.
Random Forest	Composantes : 5 NumTrees : 120 MaxDepth : 12	~1200	~5,4%	



Comparaison des valeurs réelles (en bleu) et les valeurs prédites (en vert) avec Random Forest

Sur le graphe ci-dessus, nous pouvons voir que :

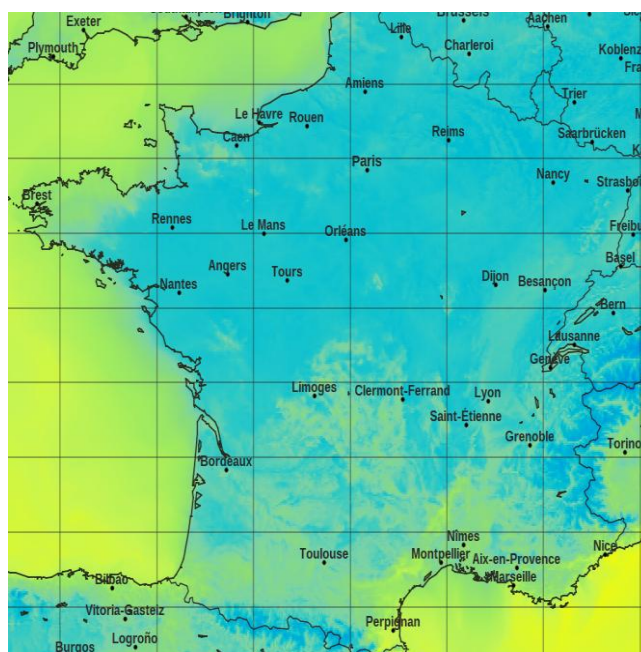
1. Même s'il existe un léger écart entre la consommation réelle et la consommation prédite, le prédicteur suit assez fidèlement les variations de la consommation réelle.
2. Très souvent, les prédictions réalisées sont inférieures aux données réelles
3. Il existe toutefois des variations que le modèle ne prend pas correctement en compte, en effet, le graphe ci-dessus nous montre que certains écarts sont plus importants que d'autre.

Données AROME et application du modèle

Cette partie présente rapidement les particularités d'AROME et les choix qui ont été opérés pour le projet.

Particularité de AROME et de son API

Le modèle AROME produit 3 types de carte au format de données météorologique grib2 (Gridded Binary) qui permettent de stocker des informations météo (températures, force du vent, ...) aux points de grilles avec une précision plus ou moins fine en fonction de la résolution (maille) de la carte.



Exemple de carte des températures au format GRIB2 produite avec AROME

Les mailles (résolution) de cartes disponibles sont les suivantes :

Carte	Caractéristiques
FRANGP0025 (0,025°) : 38N 8W 53N 12E	France – maille 2,5 km
• EURW1S40 (0,025°) : 37,5N 12W 55,4N 16E	Europe – maille 2,5 km
• EURW1S100 (0,01°) : 37,5N 12W 55,4N 16E	Europe – maille 1,3km

Les données des cartes sont générées huit fois par jours et mises à disposition aux heures suivantes :

Nom du réseau (ensemble de carte)	Heure de mise à disposition
Réseau 0h	2h45
Réseau 3h	05h45
Réseau 6h	11h05
Réseau 9h	12h30
Réseau 12h	15h45
Réseau 15h	18h10
Réseau 18h	22h45
Réseau 21h	00h30

A savoir que chacune des cartes portent sur une seule variable climatique et pour une seule et même heure, par exemple, pour obtenir la température sur les 24 prochaines heures, il faudra donc récupérer 24 cartes différentes.

Pour le projet, c'est le réseau 0h qui a été choisi, il permet de récupérer les cartes prédites avec une portée allant jusqu'à 40h.

Traitement des cartes du modèle AROME

Le format grib2 des cartes climatiques n'étant pas nativement reconnue par Spark. Elles ont été ouvertes grâce à la bibliothèque ecCodes et ensuite converties en Dataframe Spark.

La récupération des variables climatiques a ensuite été réalisée en calculant la distance entre la position GPS des capteurs climatiques et la position GPS des points de grilles avec la formule de Haversine.

$$2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right)} \right)$$

Source : [Formule de haversine — Wikipédia](#)

b- Fonction de Haversine arcsin

```
import pyspark.sql.functions as F
def distance(lat_capteur, lon_capteur, lat_arome, lon_arome):
    return F.asin(
        F.sqrt(F.pow((F.sin((F.radians(lat_arome)) - F.radians(lat_capteur))/2),2) +
        F.cos(F.radians(lat_capteur))*F.cos(F.radians(lat_arome))*
        F.pow(F.sin((F.radians(lon_arome)) - F.radians(lon_capteur))/2),2))
    ) * (2*F.lit(6371.0))
```

*Fonction créée spécialement pour calculer la distance
entre les deux coordonnées GPS*

Ce traitement prend plus ou moins longtemps car il nécessite d'ouvrir la carte, de la convertir, pour ensuite récupérer les variables climatiques de chaque station en

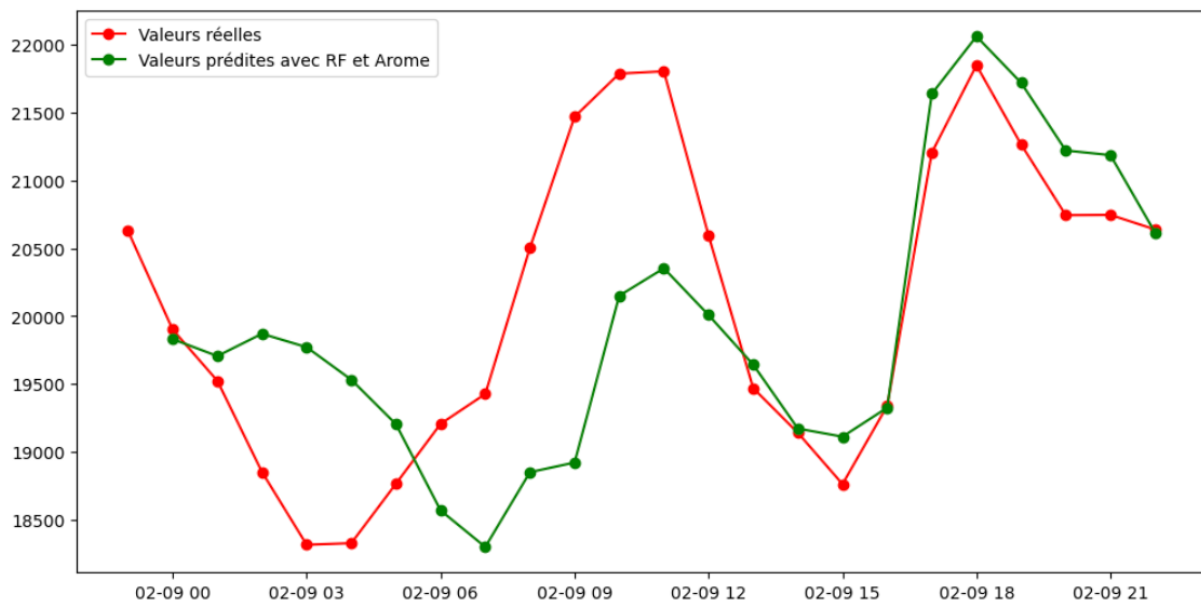
fonction des coordonnées (environ 115 stations actives de type 2 en Grand Est) et potentiellement plus de 2000 sur l'ensemble des régions.

Toutefois, la distribution de certaines opérations est envisageable afin d'éviter des goulots d'étranglement sur une seule et même machine, par exemple :

- 1- Concaténer les cartes que l'on souhaite traiter en un seul Dataframe avec l'ajout d'une colonne qui identifie chacune des cartes.
- 2- Partitionner le Dataframe en fonction de la colonne identifiant les cartes. Spark permet de réaliser cette opération avec la fonction '*répartition*'.
- 3- Procéder de manière distribuée aux opérations de calcul de distance entre la station recherchée et le point de grille de la carte le plus proche dans chaque fragment.
- 4- Retourner les variables climatiques en fonction de la distance minimum entre le point de grille et la station climatique. Cet aspect est sans doute le plus critique car il peut nécessiter des shuffles des données dans le cluster et donc des échanges réseaux potentiels.

Application du modèle et comparaison avec les données réelles

Le modèle a été appliqué pour l'instant sur peu de données mais il nous donne quand même la tendance de celui-ci et comment il peut varier par rapport aux données réelles.



Graphique de l'application du modèle sur la journée du 9 février 2025 – le MAPE est d'environ 3,8%

Sur le graphique ci-dessus de la journée du 9 février 2025, nous pouvons voir qu'il y a un léger décalage entre la prédiction du modèle et les valeurs réelles.

Nous pouvons remarquer que le modèle suit la plupart du temps assez fidèlement les variations de la consommation réelle, lorsque la consommation diminue, les

prédictions du modèle diminue et à contrario quand la consommation augmente, notre modèle augmente, sauf à l'alentour de 3h du matin.

L'erreur absolue moyenne en pourcentage est assez faible (3,8%).

Il est trop tôt pour tirer des conclusions hâtives mais les résultats sont plutôt prometteurs, toutefois, Il faudrait pour la suite du projet appliquer le modèle sur plusieurs jours et étudier si les résultats sont significatifs.

Conclusion et pistes potentielles d'amélioration

A travers ce projet, nous avons tenté de répondre à deux questions :

1. Est-ce que certaines variables climatiques ont un impact plus ou moins important sur la consommation électrique ?

Concernant la première question, les analyses effectuées ont confirmé que la consommation électrique est liée à certaines variables climatiques notamment celles des températures ou du gel.

2. La création d'un modèle permettant de prédire à partir de différentes variables dont les variables climatiques générées par les stations la consommation électrique potentielle régionale française.

La seconde question a été répondu en créant un modèle supervisé qui peut prédire la consommation électrique de la région Grand EST à partir de certaines variables climatiques et temporelles. Le projet a souhaité aller plus loin, avec l'utilisation des données atmosphérique du modèle AROME de météo France afin de prédire la consommation sur les 24 prochaines heures. Les résultats obtenus sont satisfaisants même si des axes d'améliorations existent, nous pouvons en citer quelques-uns :

1. Utiliser d'autres algorithmes de régression non-linéaire multiples comme Gradient-boosted. Jusqu'à présent, seulement les forêts aléatoires ont été utilisées.
2. Augmenter certains hyperparamètres des algorithmes déjà utilisés. Les ressources machines disponibles au cours du projet ont beaucoup limité leur utilisation.
3. Utiliser des cartes climatiques plus précises du modèle AROME et d'autres variables climatiques. Ce point reste cependant problématique car ces cartes sont relativement lourdes et donc compliquées à traiter.

Enfin, une leçon confirmée au cours du projet est l'importance d'impliquer un ou plusieurs experts métiers et de créer un lien efficace avec les data scientists. Par exemple, les tâches concernant la compréhension des différentes variables climatiques et à l'identification des correspondances possibles entre AROME et les stations climatiques ont pris un temps non négligeable. Ce temps aurait pu être amplement réduit grâce à des entretiens métiers.

Références

Spark : Valorisez vos données grâce au machine learning. Romain Jouin

Spark + Cassandra, All You Need to Know: Tips and Optimizations. Javier Ramos

Bases de données documentaires et distribuées (NFE 204)

Ingénierie de la fouille et de la visualisation de données massives (RCP 216)

Entreposage et fouille de données (STA211)

Données et documentations du modèle AROME : [Données Publiques de Météo-France - Données de modèle AROME](#)

Données et documentations des stations climatiques françaises : [Données Publiques de Météo-France - Données horaires](#)